EXPERIMENT-73(HDOC)

C Programming — Enumerations

Program 1: Enumeration for Months and the Number of Days in Each Month

Step 1: Understand the Problem Statement

- Input: A month name — JAN, FEB, MAR, ..., DEC (any letter case) — or `ALL` to list every month.
- Output: The number of days of that month — `28 or 29 days` for FEB (leap year), `30 days` for APR, JUN, SEP and NOV, and `31 days` for the other months. For ALL, every month is printed with its number of days. A name that is not a month is rejected with an error message.
- The enumeration `enum Month { JAN = 1, FEB, MAR, ..., DEC }` starts at 1, so the month constants have the values 1 to 12 that match the calendar order (JAN = 1, ..., DEC = 12); only the first value had to be written, the others follow automatically.
- `parseMonth()` converts the typed text to the enumeration value by comparing it with the array `monthNames[]` using `strcmp()`; the position i found gives the value `i + 1`.
- `daysInMonth()` receives the enumeration value and uses a `switch` statement: `case FEB` gives 28 or 29 days, the cases `APR`, `JUN`, `SEP` and `NOV` share one result of 30 days, and `default` covers the remaining seven months with 31 days.

Rule(s) used: Days in a month: FEB → 28 or 29 (29 in a leap year); APR, JUN, SEP, NOV → 30; JAN, MAR, MAY, JUL, AUG, OCT, DEC → 31. With JAN = 1, the enumeration value of a month equals its position in the year, so the array index of its name is value − 1.

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	`Input: FEB`	28 or 29 days	28 or 29 days	Pass
2	`Input: JAN`	31 days	31 days	Pass
3	`Input: APR`	30 days	30 days	Pass
4	`Input: dec`	31 days	31 days	Pass

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
5	Input: ALL	JAN = 31 days FEB = 28 or 29 days MAR = 31 days APR = 30 days MAY = 31 days JUN = 30 days JUL = 31 days AUG = 31 days SEP = 30 days OCT = 31 days NOV = 30 days DEC = 31 days	JAN = 31 days FEB = 28 or 29 days MAR = 31 days APR = 30 days MAY = 31 days JUN = 30 days JUL = 31 days AUG = 31 days SEP = 30 days OCT = 31 days NOV = 30 days DEC = 31 days	Pass
6	Input: XYZ	Invalid month. Enter JAN, FEB, ..., DEC or ALL.	Invalid month. Enter JAN, FEB, ..., DEC or ALL.	Pass

Step 3: Algorithm

1. Start the program.

2. Declare `enum Month` with JAN = 1 and the following months up to DEC, and the array `monthNames[]` with the names in the same order.

3. Read the text into `input` and change it to upper case with `toUpperString()`.

4. If the text is `ALL`, call `printAllMonths()`, which runs `month` from JAN to DEC and prints `name = days` for every month using `daysInMonth()`.

5. Otherwise call `parseMonth(input, &month)`: compare the text with every name in `monthNames[]` and, when it matches, store the enumeration value `i + 1` and return 1; return 0 if nothing matches.

6. If `parseMonth()` returns 1, call `daysInMonth(month)`, which uses a switch to return `28 or 29 days` for FEB, `30 days` for APR, JUN, SEP, NOV and `31 days` for all others, and print it.

7. If `parseMonth()` returns 0, display `Invalid month. Enter JAN, FEB, ..., DEC or ALL.`

8. Stop the program.

Evaluation against Step 2 test cases: the input FEB becomes the enumeration value 2 and the switch returns 28 or 29 days. JAN (1) falls into the default case, giving 31 days, and APR (4) is one of the four 30-day cases, giving 30 days. The lower-case input dec is changed to DEC (12) and gives 31 days. ALL prints the twelve months in order, JAN = 31 days, FEB = 28 or 29 days, MAR = 31 days, and so on up to DEC = 31 days, and the text XYZ matches no month and is rejected — identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: C Program

File: hdoc74_1.c

```
/* HDOC Day-74, Program 1: Enumeration for the months of the year.

   enum Month { JAN = 1, FEB, ..., DEC } gives JAN = 1 and every next month
   the previous value + 1 (DEC = 12). The month typed by the user (JAN to
   DEC, any letter case) is converted to the matching enumeration value and a
```

```
   switch statement gives the number of days of that month:
       FEB            -> 28 or 29 days (leap year)
       APR JUN SEP NOV -> 30 days
       all others     -> 31 days
   Typing ALL prints the number of days of every month.
*/

#include <stdio.h>
#include <string.h>
#include <ctype.h>

/* Enumeration for the months */
enum Month { JAN = 1, FEB, MAR, APR, MAY, JUN, JUL, AUG, SEP, OCT, NOV, DEC };

/* month names in the order of the enumeration values 1 to 12 */
static const char *monthNames[] = { "JAN", "FEB", "MAR", "APR", "MAY", "JUN",
                                    "JUL", "AUG", "SEP", "OCT", "NOV", "DEC" };

/* Function prototypes */
void        toUpperString(char *text);
int         parseMonth(const char *text, enum Month *month);
const char *daysInMonth(enum Month month);
void        printAllMonths(void);

int main(void) {
    char input[20];
    enum Month month;

    printf("Enter month (JAN-DEC or ALL): ");
    if (scanf("%19s", input) != 1) {
        printf("Invalid month. Enter JAN, FEB, ..., DEC or ALL.\n");
        return 1;
    }
    toUpperString(input);

    if (strcmp(input, "ALL") == 0) {
        printAllMonths();
    } else if (parseMonth(input, &month)) {
        printf("%s\n", daysInMonth(month));
    } else {
        printf("Invalid month. Enter JAN, FEB, ..., DEC or ALL.\n");
        return 1;
    }
    return 0;
}

/* Converts a string to upper case in place */
void toUpperString(char *text) {
    int i;
    for (i = 0; text[i] != '\0'; i++)
        text[i] = (char)toupper((unsigned char)text[i]);
}

/* Converts the month name to the enumeration value; returns 1 on success */
int parseMonth(const char *text, enum Month *month) {
    int i;
    for (i = 0; i < 12; i++) {
        if (strcmp(text, monthNames[i]) == 0) {
            *month = (enum Month)(i + 1);        /* JAN = 1 ... DEC = 12 */
            return 1;
        }
    }
    return 0;
}
```

```
/* Returns the number of days of the given month */
const char *daysInMonth(enum Month month) {
    switch (month) {
        case FEB:
            return "28 or 29 days";
        case APR:
        case JUN:
        case SEP:
        case NOV:
            return "30 days";
        default:
            return "31 days";
    }
}

/* Prints the number of days of every month */
void printAllMonths(void) {
    enum Month month;
    for (month = JAN; month <= DEC; month++)
        printf("%s = %s\n", monthNames[month - 1], daysInMonth(month));
}
```

Step 5: Compile, Execute and Evaluate

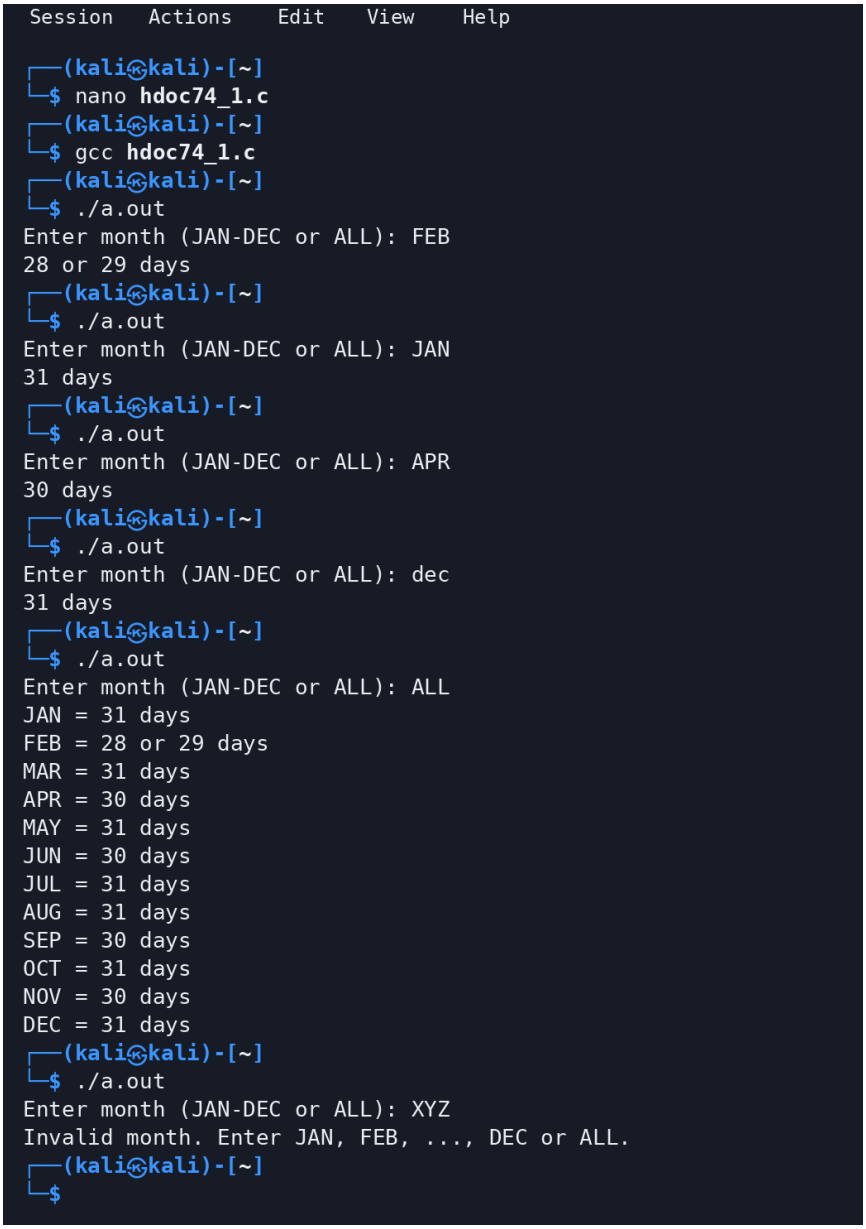
The program was compiled using the GCC compiler and executed from the Linux terminal using the following commands:

cc hdoc74_1.c	(compile)
./a.out	(execute)

All 6 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot

Screenshot 1 of 1



Program 2: Enumeration for SUCCESS, FAILURE and TIMEOUT

Step 1: Understand the Problem Statement

- Input: The status of an operation — SUCCESS, FAILURE or TIMEOUT (the letter case does not matter).
- Output: The message for that status: `Operation Successful` for SUCCESS, `Operation Failed` for FAILURE and `Operation Timed Out` for TIMEOUT. Any other text is rejected with an error message.
- The enumeration `enum Status { SUCCESS, FAILURE, TIMEOUT }` gives SUCCESS = 0, FAILURE = 1 and TIMEOUT = 2.
- `parseStatus()` converts the typed text to the matching enumeration value: the text is changed to upper case with `toupper()` and compared with the three names using `strcmp()`. It returns 1 on success and 0 for an invalid status.
- `printMessage()` receives the enumeration value and uses a `switch` statement, printing the message for `case SUCCESS`, `case FAILURE` and `case TIMEOUT`.

Rule(s) used: SUCCESS (0) → Operation Successful, FAILURE (1) → Operation Failed, TIMEOUT (2) → Operation Timed Out. The text is converted to upper case before comparing, so "timeout", "Timeout" and "TIMEOUT" all give the value 2.

Step 2: Test Case Table

Test Case	Input(s)	Expected Output(s)	Actual Output(s)	Result
1	`Input: FAILURE`	Operation Failed	Operation Failed	Pass
2	`Input: SUCCESS`	Operation Successful	Operation Successful	Pass
3	`Input: TIMEOUT`	Operation Timed Out	Operation Timed Out	Pass
4	`Input: timeout`	Operation Timed Out	Operation Timed Out	Pass
5	`Input: PENDING`	Invalid status. Enter SUCCESS, FAILURE or TIMEOUT.	Invalid status. Enter SUCCESS, FAILURE or TIMEOUT.	Pass

Step 3: Algorithm

1. Start the program.
2. Declare the enumeration `enum Status` with SUCCESS, FAILURE and TIMEOUT (values 0, 1 and 2).
3. Read the status text into `input`.
4. Call `parseStatus(input, &status)`: copy the text in upper case and compare it with "SUCCESS", "FAILURE" and "TIMEOUT" using `strcmp()`; store the matching constant in `status` and return 1, or return 0 if nothing matches.

5. If `parseStatus()` returns 0 (or nothing was read), display `Invalid status. Enter SUCCESS, FAILURE or TIMEOUT.` and stop.

6. Call `printMessage(status)`.

7. In `printMessage()`: print `Operation Successful` for SUCCESS, `Operation Failed` for FAILURE and `Operation Timed Out` for TIMEOUT.

8. Stop the program.

Evaluation against Step 2 test cases: the input FAILURE is converted to the enumeration value 1, so the switch prints Operation Failed. SUCCESS (value 0) prints Operation Successful and TIMEOUT (value 2) prints Operation Timed Out. The lower-case input timeout becomes TIMEOUT after conversion and gives the same message, while PENDING matches none of the three names and is rejected with the error message — identical to the Expected Output(s) column, confirming the algorithm is correct before coding.

Step 4: C Program

File: hdoc74_2.c

```c
/* HDOC Day-74, Program 2: Enumeration for the status of an operation.

   enum Status { SUCCESS, FAILURE, TIMEOUT } gives SUCCESS = 0, FAILURE = 1
   and TIMEOUT = 2. The status typed by the user is converted to the matching
   enumeration value and a switch statement prints the message for it:
       SUCCESS -> Operation Successful
       FAILURE -> Operation Failed
       TIMEOUT -> Operation Timed Out
*/

#include <stdio.h>
#include <string.h>
#include <ctype.h>

/* Enumeration for the status of an operation */
enum Status { SUCCESS, FAILURE, TIMEOUT };

/* Function prototypes */
int  parseStatus(const char *text, enum Status *status);
void printMessage(enum Status status);

int main(void) {
    char input[20];
    enum Status status;

    printf("Enter status (SUCCESS, FAILURE, TIMEOUT): ");
    if (scanf("%19s", input) != 1 || !parseStatus(input, &status)) {
        printf("Invalid status. Enter SUCCESS, FAILURE or TIMEOUT.\n");
        return 1;
    }

    printMessage(status);
    return 0;
}

/* Converts the typed status (any letter case) to the enumeration value.
   Returns 1 on success and 0 if the text is not a valid status. */
int parseStatus(const char *text, enum Status *status) {
    char upper[20];
    int i;

    for (i = 0; text[i] != '\0' && i < 19; i++)
```

```
        upper[i] = (char)toupper((unsigned char)text[i]);
    upper[i] = '\0';

    if (strcmp(upper, "SUCCESS") == 0)
        *status = SUCCESS;
    else if (strcmp(upper, "FAILURE") == 0)
        *status = FAILURE;
    else if (strcmp(upper, "TIMEOUT") == 0)
        *status = TIMEOUT;
    else
        return 0;
    return 1;
}

/* Prints the message that belongs to the value of the enumeration */
void printMessage(enum Status status) {
    switch (status) {
        case SUCCESS:
            printf("Operation Successful\n");
            break;
        case FAILURE:
            printf("Operation Failed\n");
            break;
        case TIMEOUT:
            printf("Operation Timed Out\n");
            break;
    }
}
```

Step 5: Compile, Execute and Evaluate

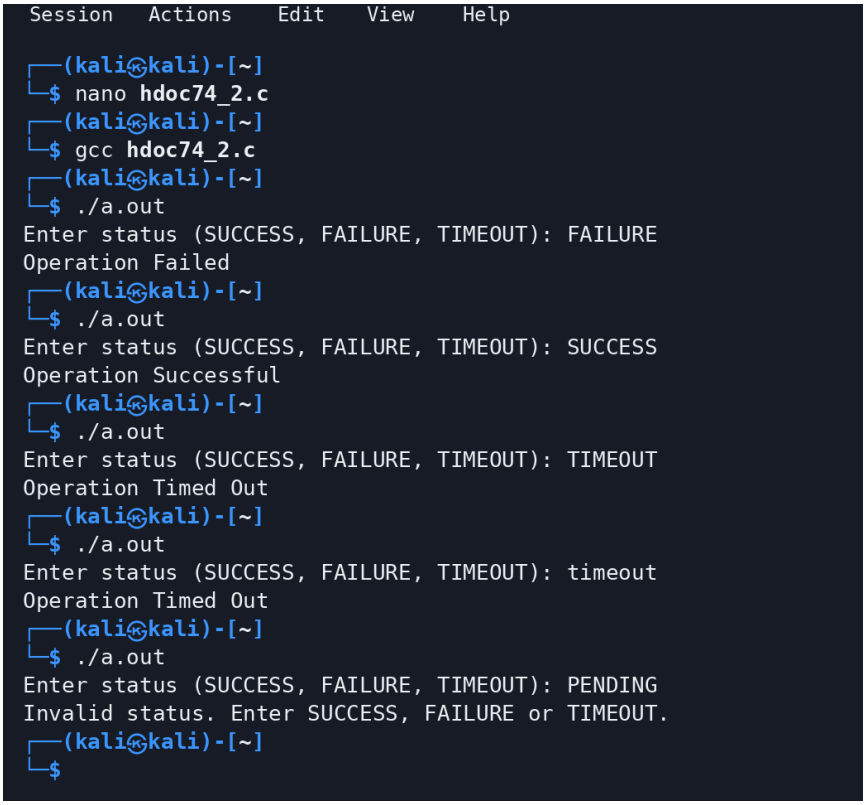
The program was compiled using the GCC compiler and executed from the Linux terminal using the following commands:

`cc hdoc74_2.c`	(compile)
`./a.out`	(execute)

All 5 test cases prepared in Step 2 were executed against the compiled program. The Actual Output(s) obtained matched the Expected Output(s) in every case, so all test cases have been marked as Pass (see table in Step 2).

Step 6: Compilation and Execution Screenshot

Screenshot 1 of 1



Conclusion

In this experiment, enumerations were used to give meaningful names to integer constants. In the first program an enum for the months (JAN = 1 to DEC = 12) was declared, the typed month was converted to its enum value, and a switch statement printed the number of days of the month (28 or 29 for FEB, 30 or 31 for the others), with an ALL option that lists every month. In the second program an enum with SUCCESS, FAILURE and TIMEOUT (0, 1 and 2) was used: the typed status was converted to its enum value and a switch statement printed Operation Successful, Operation Failed or Operation Timed Out. Both programs were compiled with gcc and tested, including lower-case and invalid inputs, and every output matched the expected result.